

100 Days of Linux - Day 069

Title: Mini Project - Interactive Bash Application

Today's Goal

Combine variables, user input, command-line arguments, conditions, loops, and functions to build a complete interactive Bash application.

Why This Matters

Real Bash automation combines multiple concepts. This project mirrors the structure of scripts used by Linux administrators, DevOps engineers, and data engineers.

Concepts Used

Variables, read, command-line arguments, if/elif/else, for loops, while loops, functions

Project Scenario

Build a script named **student_manager.sh**. The script should greet the user, ask for student details, validate simple input, display a summary, and print a numbered list of tasks using loops and functions.

Practice Questions (Exactly 5)

1. Create `student_manager.sh` with a shebang and a `greet()` function that displays a welcome message.
2. Ask the user for Name, Course, and City using the `read` command. Store each value in variables.
3. Accept an optional command-line argument representing the training batch. If no argument is provided, display 'Default Batch'.
4. Use an `if` statement to check whether the Course entered is Linux100. Print an appropriate message. Then use a `for` loop to display five learning milestones.
5. Create a `summary()` function that prints all collected information, run the script, and display your current working directory.

Hints

Break the script into small functions. Test each part before combining everything.

Mini Challenge

Extend the script by adding a while loop menu with options to display the summary, restart data entry, or exit.

Common Mistakes

Trying to build the entire script at once instead of testing one feature at a time.

Did You Know?

Many deployment scripts use functions, loops, and conditions together to automate complex workflows.

Pro Tip

Write reusable functions such as `greet()`, `collect_input()`, and `summary()` to keep scripts clean.

Answer Key (Instructor Only)

Q1: Create `greet()` and call it.

Q2: Use `read NAME`, `read COURSE`, `read CITY`.

Q3: Use `$1` with an `if` statement to handle a missing argument.

Q4: Use `if [ "$COURSE" = "Linux100" ]` and a `for` loop.

Q5: Create `summary()` and execute the script, then run `pwd`.

Homework

Enhance `student_manager.sh` by saving the summary to a text file and allowing multiple students to be entered using a loop.

Next Day Preview

Tomorrow you'll complete the Shell Scripting module with a comprehensive practical assignment covering everything learned from Days 62–69.